System Calls

External, Internal, and software Interrupts

External interrupts (hardware interrupts) Asynchronous	Internal interrupts (exceptions) Synchronous	Software interrupts (traps) Synchronous
Examples: Moving mouse, or pressing a key on the keyboard	Generated by CPU itself whenever some unusual condition occurs **Examples:** 1/0	**Examples:** System calls

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Two main functions of operating systems

Providing abstractions to user programs and managing the computer's resources.

The interaction between user programs and the operating system mainly involves managing abstractions, while resource management is handled automatically and is mostly invisible to the users.

OS provides the set of system calls that is an interface between the operating system and the user programs.

1. The system calls vary from operating system to operating system.

2. System calls are highly machine dependent and often must be expressed in assembly code. So an OS provides a procedure library to make system calls from C programs and often from other languages as well.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux Programmer's Manual

```
1   Executable programs or shell commands
2   System calls (functions provided by the kernel)
3   Library calls (functions within program libraries)
4   Special files (usually found in /dev)
5   File formats and conventions, e.g. /etc/passwd
6   Games
7   Miscellaneous (including  macro  packages  and  conventions),  e.g.
    man(7), groff(7)
8   System administration commands (usually only for root)
9   Kernel routines [Non standard]
```

```
FORK(2)

NAME
       fork - create a child process

SYNOPSIS
       #include <sys/types.h>
       #include <unistd.h>

       pid_t fork(void);
```

```
SUDO(8)

NAME
     sudo, sudoedit -
```

```
LS(1)

NAME
       ls - list directory contents
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux Programmer's Manual

```
1   Executable programs or shell commands
2   System calls (functions provided by the kernel)
3   Library calls (functions within program libraries)
4   Special files (usually found in /dev)
5   File formats and conventions, e.g. /etc/passwd
6   Games
7   Miscellaneous (including  macro  packages  and  conventions),  e.g.
    man(7), groff(7)
8   System administration commands (usually only for root)
9   Kernel routines [Non standard]
```

```
FORK(2)

NAME
       fork - create a child process

SYNOPSIS
       #include <sys/types.h>
       #include <unistd.h>

       pid_t fork(void);
```

```
SUDO(8)

NAME
       sudo, sudoedit -
```

```
LS(1)

NAME
       ls - list directory contents
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Standard streams in Linux

	symbol	fd	Examples	
standard input (stdin)	<	0	keyboard or file	> cat > cat < myfile.txt
standard output (stdout)	1>	1	terminal or file	> cat 1> myfile.txt or cat > myfile.txt
standard error (stderr)	2>	2	terminal or file	> cat myfile > cat myfile 2> log_error

System Calls

Standard streams in Linux

```c
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
  int fd=0, size;
  char *buffer = (char *) calloc(100, sizeof(char));
  size = read(fd, buffer, 100);
  printf("called read(% d, buffer, 100). returned that %d bytes were read.\n", fd, size);
  buffer[size] = '\0';
  printf("Those bytes are as follows: %s \n", buffer);
}

```

System Calls

Standard streams in Linux

```
~$
~$ ./a.out
Hello World
called read( 0, buffer, 100). returned that 13 bytes were read.
Those bytes are as follows: Hello World

~$
~$ ./a.out < myFile
called read( 0, buffer, 100). returned that 21 bytes were read.
Those bytes are as follows: Hello from my file.
```

System Calls

Standard streams in Linux

```c
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(){
  int fd=0, size;
  char *buffer = (char *) calloc(100, sizeof(char));
  size = read(fd, buffer, 100);
  printf("called read(% d, buffer, 100). returned that %d bytes were read.\n", fd, size);
  buffer[size] = '\0';
  printf("Those bytes are as follows: %s \n", buffer);
  write(1, buffer, size);
}
```

System Calls

Standard streams in Linux

```
~$
~$
~$ ./a.out
Hello World
called read( 0, buffer, 100). returned that 12 bytes were read.
Those bytes are as follows: Hello World

Hello World
~$
~$
~$ ./a.out < myFile
called read( 0, buffer, 100). returned that 21 bytes were read.
Those bytes are as follows: Hello from my file.

Hello from my file.
~$
```

System Calls

Linux System Calls
Assembly Language(x86)

- 8 bit registers : A, B, C, D, ...
- 16 bit registers : AX, BX, CX, DX, . . .
- 32 bit registers : EAX, EBX, ECX, EDX, . . .
- 64 bit registers : RAX, RBX, RCX, RDX, . . .

```
AX=0000  BX=0000  CX=0000  DX=0000  SP=FFEE  BP=0000  SI=0000  DI=0000
DS=1165  ES=1165  SS=1165  CS=1165  IP=0100   NV UP EI PL NZ NA PO NC
1165:0100 0F              DB      0F
```

AX = 0000h(16 bits)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls
Assembly Language(x86)

1. System call number: EAX register.
2. Arguments to the system call : registers EBX, ECX, etc.
3. Invoke the relevant interrupt (e.g. 80h is used for kernel call).
4. The result, usually returned in the EAX register.

eax	Name	ebx	ecx	edx	esx	edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls
Assembly Language(x86)

```
section .text
    global _start     ;must be declared for using gcc
_start:               ;tell linker entry point
    mov edx, len      ;message length
    mov ecx, msg      ;message to write
    mov ebx, 1        ;file descriptor (stdout)
    mov eax, 4        ;system call number (sys_write)
    int 0x80          ;call kernel
    mov eax, 1        ;system call number (sys_exit)
    int 0x80          ;call kernel

section .data

msg db  'Hello, world!', 0xa     ;our dear string
len equ $ - msg          ;length of our dear string
```

eax	Name	ebx	ecx	edx	esx	edi
1	sys_exit	int	-	-	-	-
4	sys_write	unsigned int	const char *	size_t	-	-

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls
Assembly Language(x86)

```
section .text
    global _start     ;must be declared for using gcc
_start:               ;tell linker entry point
    mov edx, len      ;message length
    mov ecx, msg      ;message to write
    mov ebx, 1        ;file descriptor (stdout)
    mov eax, 4        ;system call number (sys_write)
    int 0x80          ;call kernel
    mov eax, 1        ;system call number (sys_exit)
    int 0x80          ;call kernel

section .data

msg db  'Hello, world!', 0xa     ;our dear string
len equ $ - msg          ;length of our dear string
```

```
~$ nasm -felf  hello.asm
~$ ld -m elf_i386  -o hello hello.o
~$ ./hello
Hello, world!
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls

man syscall

The first table lists the instruction used to transition to kernel mode (which might not be the fastest or best way to transition to the kernel, so you might have to refer to **vdso**(7)), the register used to indicate the system call number, the register(s) used to return the system call result, and the register used to signal an error.

Arch/ABI	Instruction	System call #	Ret val	Ret val2	Error	Notes
alpha	callsys	v0	v0	a4	a3	1, 6
arc	trap0	r8	r0	-	-	
arm/OABI	swi NR	-	r0	-	-	2
arm/EABI	swi 0x0	r7	r0	r1	-	
arm64	svc #0	w8	x0	x1	-	
blackfin	excpt 0x0	P0	R0	-	-	
i386	int $0x80	eax	eax	edx	-	
ia64	break 0x100000	r15	r8	r9	r10	1, 6
m68k	trap #0	d0	d0	-	-	
microblaze	brki r14,8	r12	r3	-	-	
mips	syscall	v0	v0	v1	a3	1, 6
nios2	trap	r2	r2	-	r7	
parisc	ble 0x100(%sr2, %r0)	r20	r28	-	-	
powerpc	sc	r0	r3	-	r0	1
powerpc64	sc	r0	r3	-	cr0.SO	1
riscv	ecall	a7	a0	a1	-	
s390	svc 0	r1	r2	r3	-	3
s390x	svc 0	r1	r2	r3	-	3
superh	trap #0x17	r3	r0	r1	-	4, 6
sparc/32	t 0x10	g1	o0	o1	psr/csr	1, 6
sparc/64	t 0x6d	g1	o0	o1	psr/csr	1, 6
tile	swint1	R10	R00	-	R01	1
x86-64	syscall	rax	rax	rdx	-	5
x32	syscall	rax	rax	rdx	-	5
xtensa	syscall	a2	a2	-	-	

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls

The second table shows the registers used to pass the system call arguments.

Arch/ABI	arg1	arg2	arg3	arg4	arg5	arg6	arg7	Notes
alpha	a0	a1	a2	a3	a4	a5	-	
arc	r0	r1	r2	r3	r4	r5	-	
arm/OABI	r0	r1	r2	r3	r4	r5	r6	
arm/EABI	r0	r1	r2	r3	r4	r5	r6	
arm64	x0	x1	x2	x3	x4	x5	-	
blackfin	R0	R1	R2	R3	R4	R5	-	
i386	ebx	ecx	edx	esi	edi	ebp	-	
ia64	out0	out1	out2	out3	out4	out5	-	
m68k	d1	d2	d3	d4	d5	a0	-	
microblaze	r5	r6	r7	r8	r9	r10	-	
mips/o32	a0	a1	a2	a3	-	-	-	1
mips/n32,64	a0	a1	a2	a3	a4	a5	-	
nios2	r4	r5	r6	r7	r8	r9	-	
parisc	r26	r25	r24	r23	r22	r21	-	
powerpc	r3	r4	r5	r6	r7	r8	r9	
powerpc64	r3	r4	r5	r6	r7	r8	-	
riscv	a0	a1	a2	a3	a4	a5	-	
s390	r2	r3	r4	r5	r6	r7	-	
s390x	r2	r3	r4	r5	r6	r7	-	
superh	r4	r5	r6	r7	r0	r1	r2	
sparc/32	o0	o1	o2	o3	o4	o5	-	
sparc/64	o0	o1	o2	o3	o4	o5	-	
tile	R00	R01	R02	R03	R04	R05	-	
x86-64	rdi	rsi	rdx	r10	r8	r9	-	
x32	rdi	rsi	rdx	r10	r8	r9	-	
xtensa	a6	a3	a4	a5	a8	a9	-	

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls

```
asp@asp-VB:~$ ausyscall --dump
Using x86_64 syscall table:
0       read
1       write
2       open
3       close
4       stat
5       fstat
6       lstat
7       poll
8       lseek
9       mmap
10      mprotect
11      munmap
12      brk
13      rt_sigaction
14      rt_sigprocmask
15      rt_sigreturn
16      ioctl
17      pread
18      pwrite
19      readv
20      writev
21      access
22      pipe
23      select
24      sched_yield
25      mremap
430     fsopen
431     fsconfig
432     fsmount
433     fspick
434     pidfd_open
435     clone3
436     close_range
437     openat2
438     pidfd_getfd
439     faccessat2
440     process_madvise
441     epoll_pwait2
442     mount_setattr
443     quotactl_fd
444     landlock_create_ruleset
445     landlock_add_rule
446     landlock_restrict_self
447     memfd_secret
448     process_mrelease
449     futex_waitv
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls
Assembly Language(MIPS)

```
.data
        my_string: .asciiz "\nHello, World!\n"

.text
        main:
                li $v0, 15
                la $a0, 1
                la $a1, my_string
                la $a2, 15
                syscall

                li $v0, 10
                syscall
```

```
Hello, World!
```

write to file 15

$a0 = file descriptor
$a1 = address of output buffer
$a2 = number of characters to write

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Linux System Calls

DESCRIPTION

syscall() is a small library function that invokes the system call whose assembly language interface has the specified number with the specified arguments. Employing syscall() is useful, for example, when invoking a system call that has no wrapper function in the C library.

```c
#include <unistd.h>
#include <sys/syscall.h>
#include <stdio.h>

#define STDIN_FILENO 0
#define BUFFER_SIZE 100

int main() {
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead;

    printf("Enter your message: ");
    fflush(stdout);
    bytesRead = syscall(SYS_read, STDIN_FILENO, buffer, BUFFER_SIZE - 1);
    if (bytesRead == -1) {
        perror("Read error");
        return 1;
    }
    buffer[bytesRead] = '\0';
    printf("You message is : %s\n", buffer);
    return 0;
}
```

```
Enter your message: Hello world!
You message is : Hello world!
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

read(fd, buffer, nbytes)

System calls are performed in a series of steps

Steps 1,2, and 3: pushing the parameters onto the stack in reverse order.

Step 4: calling to the library procedure. This instruction is the normal procedure-call instruction used to call all procedures.

Step 5: The library procedure, possibly written in assembly language, typically puts the system-call number in a place where the operating system expects it, such as a register.

Step 6: executing a TRAP instruction to switch from user mode to kernel mode and start execution at a fixed address within the kernel.

Step 7: The kernel code that starts following the TRAP examines the system-call number and then dispatches to the correct system-call handler, usually via a table of pointers to system-call handlers indexed on system-call number.

System Calls

Step 8: At that point the system-call handler runs.

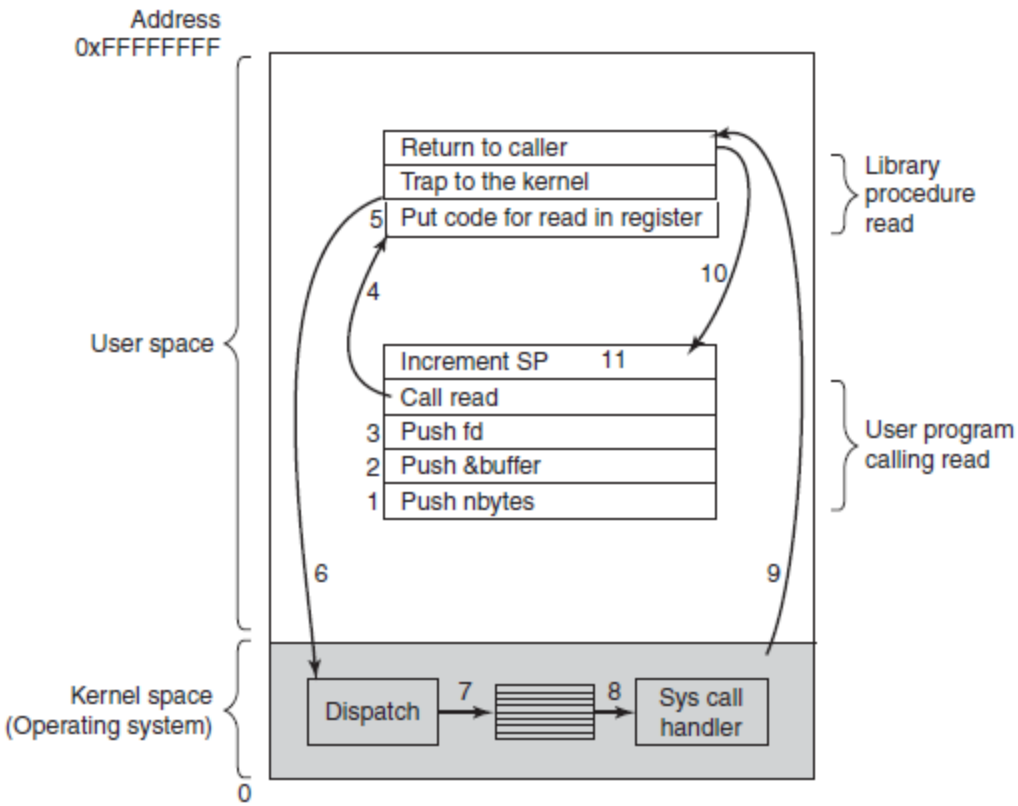
Step 9: Once it has completed its work, control may be returned to the user-space library procedure at the instruction following the TRAP instruction.

Step 10: This procedure then returns to the user program in the usual way procedure calls return.

Step 11: To finish the job, the user program has to clean up the stack, as it does after any procedure call.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls



The 11 steps in making the system call *read(fd, buffer, nbytes).*

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Some of the major POSIX system calls. The return code *s* is −1 if an error has occurred. The return codes are as follows: *pid* is a process id, *fd* is a file descriptor, *n* is a byte count, *position* is an offset within the file, and *seconds* is the elapsed time.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

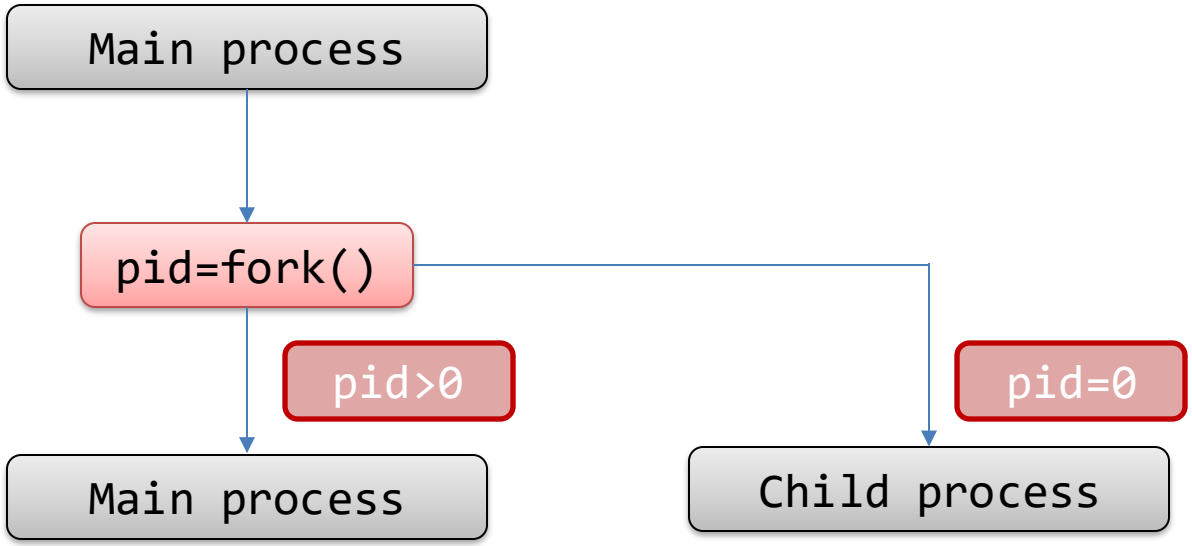
```
pid=-1  fork fails
pid=0   return value in the child process
pid>0   return value in the parent process
        (returns the process ID of the child process to the parent process)
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Successful fork (fork()≥0):

<u>Execution in both the parent and child process continues at the instruction following the fork.</u>

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    printf("Line number : %d  \n", __LINE__);
    pid_t pid;
    pid = fork();
    printf("Line number : %d , pid = %d \n", __LINE__, pid);
    printf("Line number : %d , pid = %d \n", __LINE__, pid);

    return 0;
}
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  int main()
5  {
6      printf("Line number : %d  \n", __LINE__);
7      pid_t pid;
8      pid = fork();
9      printf("Line number : %d , pid = %d \n", __LINE__, pid);
10     printf("Line number : %d , pid = %d \n", __LINE__, pid);
11
12     return 0;
13 }
```

```
~$ ./a.out
Line number : 6
Line number : 9  , pid = 736
Line number : 10 , pid = 736
Line number : 9  , pid = 0
Line number : 10 , pid = 0
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    printf("Line number : %d  \n
    pid_t pid;
    pid = fork();
    printf("Line number : %d , pid = %d \n", __LINE__, pid);
    printf("Line number : %d , pid = %d \n", __LINE__, pid);

    return 0;
}
```

```
~$ ./a.out
Line number : 6
Line number : 9 , pid = 736
Line number : 10 , pid = 736
Line number : 9 , pid = 0
Line number : 10 , pid = 0
~$
```

Parent

Child

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    printf("Line number : %d  \n", __LINE__);
    pid_t pid;
    pid = fork();
    if (pid==0){
        printf("Line number : %d  , pid = %d \n", __LINE__, pid);
        return 0;
    }
    printf("Line number : %d , pid = %d \n", __LINE__, pid);

    return 0;
}
```

```
~$ ./a.out
```

?

System Calls

```c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  int main()
5  {
6      printf("Line number : %d  \n", __LINE__);
7      pid_t pid;
8      pid = fork();
9      if (pid==0){
10         printf("Line number : %d  , pid = %d \n", __LINE__, pid);
11         return 0;
12     }
13     printf("Line number : %d , pid = %d \n", __LINE__, pid);
14
15     return 0;
16 }
```

```
~$ ./a.out
Line number : 6
Line number : 13 , pid = 984
Line number : 10  , pid = 0
~$
```

System Calls

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    printf("Line number : %d  \n", __LINE__);
    pid_t pid;
    pid = fork();
    if (pid==0){
        printf("Line number : %d  , pid = %d \n", __LINE__, pid);
        return 0;
    }
    printf("Line number : %d , pid = %d \n", __LINE__, pid);

    return 0;
}
```

```
~$ ./a.out
Line number : 6
Line number : 13 , pid = 984
Line number : 10  , pid = 0
~$
```

Parent

Child

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4
5  int main()
6  {
7      int status;
8      pid_t fork_return = fork();
9      if (fork_return == 0){
10         int sum = 0;
11         for(int i=1; i<101; i++){
12             sum += i;
13             }
14         printf("child done : sum = %d \n", sum);
15     }
16     else{
17         printf("Line : %d \n", __LINE__);
18         waitpid(-1, &status, 0);
19     }
20     printf("Line : %d , Fork return = %d \n", __LINE__, fork_return);
21     return 0;
22 }
```

```
~$ ./a.out
Line : 17
child done : sum = 5050
Line : 20 , Fork return = 0
Line : 20 , Fork return = 1825
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```
EXECVE(2)                    Linux Programmer's Manual

NAME
       execve - execute program
```

```
EXEC(3)                      Linux Programmer's Manual

NAME
       execl, execlp, execle, execv, execvp, execvpe - execute a file
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```
#include <unistd.h>
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

DESCRIPTION

execve() executes the program referred to by pathname. This causes the program that is currently being run by the calling process to be replaced with a new program, with newly initialized stack, heap, and data segments.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```
EXECVE(2)                    Linux Programmer's Manual

NAME
       execve - execute program
```

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char* argv[]){
    static char *args[] = { "", "-al","-d", NULL };
    execve("/usr/bin/ls",args,0);
    printf("Error \n");
    return 0;
}
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```
EXECVE(2)                    Linux Programmer's Manual

NAME
        execve - execute program
```

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char* arg
    static char *args[] = {
    execve("/usr/bin/ls",arg
    printf("Error \n");
    return 0;
}
```

```
~$ ./a.out
drwxr-xr-x 4 user user 21 May 17 04:55 .
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

```
EXECVE(2)                    Linux Programmer's Manual

NAME
        execve - execute program
```

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main(int argc, char* arg
    static char *args[] = {
    execve("/usr/bin/ls",arg
    printf("Error \n");
    return 0;
}
```

?

```
~$ ./a.out
drwxr-xr-x 4 user user 21 May 17 04:55 .
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

```
#define TRUE 1

while (TRUE) {                                  /* repeat forever */
        type_prompt( );                         /* display prompt on the screen */
        read_command(command, parameters);      /* read input from terminal */

        if (fork( ) != 0) {                     /* fork off child process */
            /* Parent code. */
            waitpid(-1, &status, 0);            /* wait for child to exit */
        } else {
            /* Child code. */
            execve(command, parameters, 0);     /* execute command */
        }
}
```

A stripped-down shell. Throughout this book, *TRUE* is assumed to be defined as 1.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing, or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Some of the major POSIX system calls. The return code *s* is −1 if an error has occurred. The return codes are as follows: *pid* is a process id, *fd* is a file descriptor, *n* is a byte count, *position* is an offset within the file, and *seconds* is the elapsed time.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Directory and file system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Some of the major POSIX system calls. The return code *s* is −1 if an error has occurred. The return codes are as follows: *pid* is a process id, *fd* is a file descriptor, *n* is a byte count, *position* is an offset within the file, and *seconds* is the elapsed time.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls

Miscellaneous

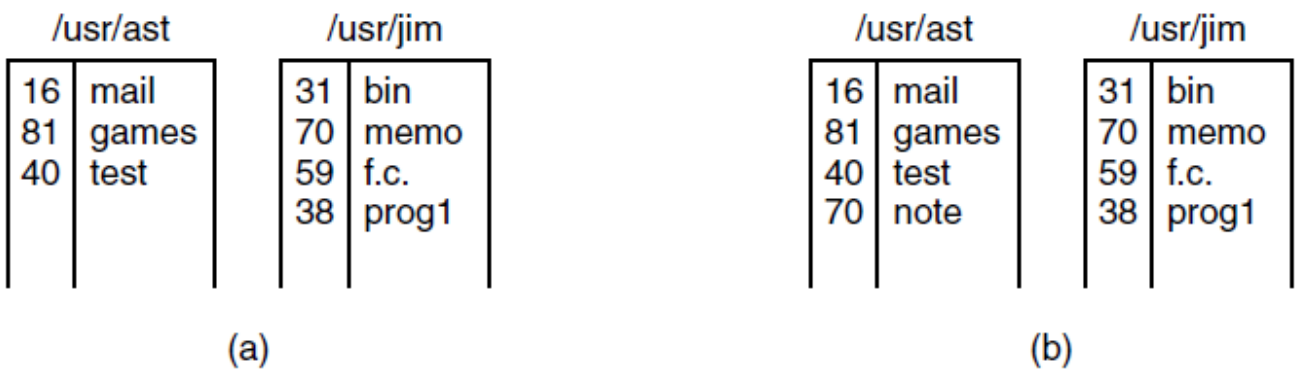
Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

Some of the major POSIX system calls. The return code *s* is −1 if an error has occurred. The return codes are as follows: *pid* is a process id, *fd* is a file descriptor, *n* is a byte count, *position* is an offset within the file, and *seconds* is the elapsed time.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

System Calls for File Management

```
link("/usr/jim/memo", "/usr/ast/note");
```



(a) Two directories before linking *usr/jim/memo* to *ast*'s directory.

(b) The same directories after linking.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Shared Files
Hard link

# of links to the main file

```
.
├── my_file
├── my_file_h_link
└── my_file_s_link -> my_file

0 directories, 3 files

  File: my_file
  Size: 29              Blocks: 1          IO Block: 65536  regular file
Device: 20318e08h/540118536d    Inode: 20829148276976248  Links: 2
Access: (0644/-rw-r--r--)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-02 22:19:33.473215500 -0700
Modify: 2020-08-02 22:19:33.484830600 -0700
Change: 2020-08-02 22:20:48.981441800 -0700
 Birth: 2020-08-02 22:19:33.473215500 -0700

  File: my_file_h_link
  Size: 29              Blocks: 1          IO Block: 65536  regular file
Device: 20318e08h/540118536d    Inode: 20829148276976248  Links: 2
Access: (0644/-rw-r--r--)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-02 22:19:33.473215500 -0700
Modify: 2020-08-02 22:19:33.484830600 -0700
Change: 2020-08-02 22:20:48.981441800 -0700
 Birth: 2020-08-02 22:19:33.473215500 -0700
```

Shared Files
Symbolic link

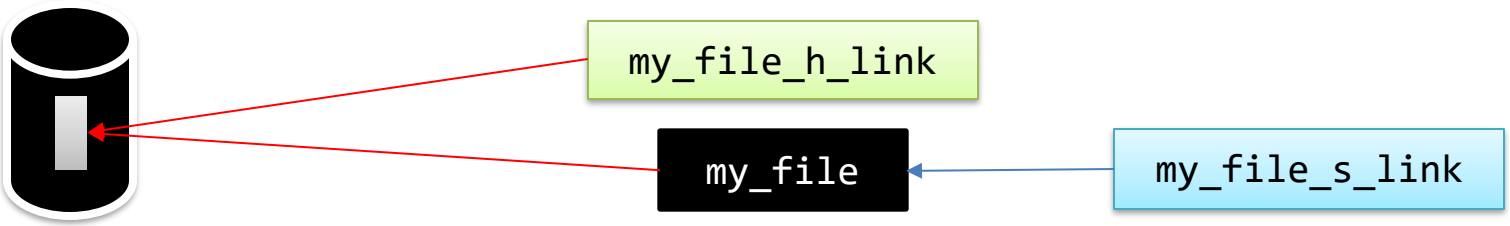
```
.
├── my_file
├── my_file_h_link
└── my_file_s_link -> my_file

0 directories, 3 files

  File: my_file
  Size: 29              Blocks: 1          IO Block: 65536  regular file
Device: 20318e08h/540118536d    Inode: 20829148276976248  Links: 2
Access: (0644/-rw-r--r--)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-02 22:19:33.473215500 -0700
Modify: 2020-08-02 22:19:33.484830600 -0700
Change: 2020-08-02 22:20:48.981441800 -0700
 Birth: 2020-08-02 22:19:33.473215500 -0700

  File: my_file_s_link -> my_file
  Size: 7               Blocks: 0          IO Block: 65536  symbolic link
Device: 20318e08h/540118536d    Inode: 6192449488013181  Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-01 21:18:56.790120800 -0700
Modify: 2020-08-01 21:18:56.790120800 -0700
Change: 2020-08-01 21:18:56.790120800 -0700
 Birth: 2020-08-01 21:18:56.790120800 -0700
```

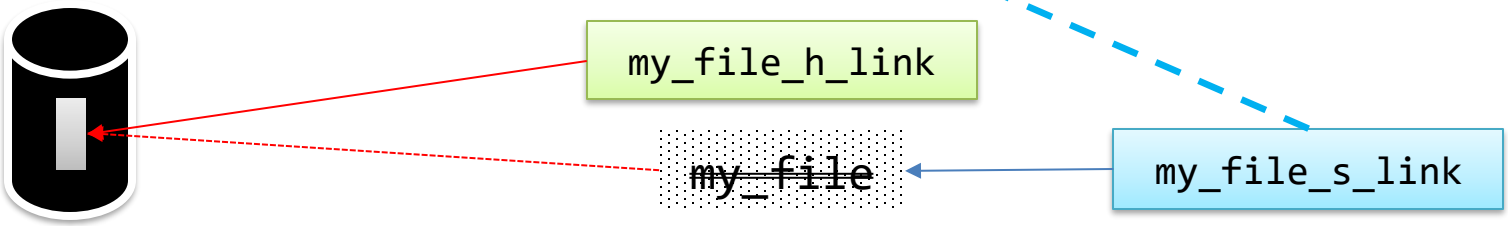
Shared Files



Shared Files Deleting

```
  File: my_file_h_link
  Size: 29              Blocks: 1          IO Block: 65536  regular file
Device: 20318e08h/540118536d    Inode: 20829148276976248  Links: 1
Access: (0644/-rw-r--r--)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-02 22:19:33.473215500 -0700
Modify: 2020-08-02 22:19:33.484830600 -0700
Change: 2020-08-02 22:45:24.746689100 -0700
 Birth: 2020-08-02 22:19:33.473215500 -0700

  File: my_file_s_link -> my_file
  Size: 7               Blocks: 0          IO Block: 65536  symbolic link
Device: 20318e08h/540118536d    Inode: 6192449488013181  Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (2739586/A0)          Gid: (1049089/AD+Group(513))
Access: 2020-08-01 21:18:56.790120800 -0700
Modify: 2020-08-01 21:18:56.790120800 -0700
Change: 2020-08-01 21:18:56.790120800 -0700
 Birth: 2020-08-01 21:18:56.790120800 -0700
OS:>> cat my_file_h_link
Link test
I added  second line.
OS:>> cat my_file_s_link
cat: my_file_s_link: No such file or directory
```



System Calls for File Management

As we have mentioned earlier, the mount system call allows two file systems to be merged into one. A common situation is to have the root file system, containing the binary (executable) versions of the common commands and other heavily used files, on a hard disk (sub)partition and user files on another (sub)partition. Further, the user can then insert a USB disk with files to be read.

(a) File system before the mount.
(b) File system after the mount.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Windows Win32 API

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

The Windows Win32 API

lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Linux

```
                                                        General Commands Manual
STRACE(1)
NAME
       strace - trace system calls and signals
```

```
Statistics
    -c
    --summary-only
            Count time, calls, and errors for each system call and report a summary on program  exit,  suppressing  the  regular
            output.   This  attempts  to show system time (CPU time spent running in the kernel) independent of wall clock time.
            If -c is used with -f, only aggregate totals for all traced processes are kept.
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Linux

```c
#include <time.h>
#include <unistd.h>
#include <stdio.h>

int main(){
  printf("Hello world!\n");
  return 0;
}

```

```
~$ strace  -c ./a.out
Hello world!
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         1           read
  0.00    0.000000           0         1           write
  0.00    0.000000           0         2           close
  0.00    0.000000           0         3           fstat
  0.00    0.000000           0         7           mmap
  0.00    0.000000           0         3           mprotect
  0.00    0.000000           0         1           munmap
  0.00    0.000000           0         3           brk
  0.00    0.000000           0         6           pread64
  0.00    0.000000           0         1         1 access
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         2         1 arch_prctl
  0.00    0.000000           0         2           openat
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000                    33         2 total
~$
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

System calls are the mechanism by which programs interact with the kernel of an operating system.

In the context of file I/O operations, system calls like open, read, and write play a critical role.

These operations can be executed in either blocking or non-blocking modes.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

A blocking system call suspends the execution of the process until the operation is completed.

For example, if you are reading from a file, the program will wait (or block) until the read operation is finished. Most I/O system calls, by default, are blocking calls.

Example:

In the example below, the open call blocks the process until the file is opened, which means that the program execution is paused until the operation completes.

```
int fd = open("file.txt", O_RDONLY);  // Blocking call
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

A **non-blocking system call** returns immediately, even if the requested operation hasn't been completed. Instead of waiting, the program continues executing other code while periodically checking if the operation has finished.

Non-blocking calls are valuable in scenarios where waiting for I/O operations—such as reading from a network socket or file—would lead to inefficient resource utilization.

To enable non-blocking mode, you often use the fcntl system call to modify the file descriptor.

Example:

```
#include <fcntl.h>


int fd = open("file.txt", O_RDONLY);
fcntl(fd, F_SETFL, O_NONBLOCK);  // Set the file descriptor to non-blocking mode
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

Asynchronous I/O (POSIX AIO)

POSIX Asynchronous I/O (AIO) provides another method of non-blocking I/O where you can initiate I/O operations and have them complete asynchronously, i.e., in the background.

This allows a program to perform other tasks while waiting for the operation to complete.

Functions like **aio_read**, **aio_write**, **aio_error**, **aio_return** and **aio_fsync** enable asynchronous operations.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

Asynchronous I/O (POSIX AIO)

Example:

```
#include <aio.h>

struct aiocb cb;
int fd = open("file.txt", O_RDONLY);

cb.aio_fildes = fd;
cb.aio_buf = buffer;
cb.aio_nbytes = sizeof(buffer);
cb.aio_offset = 0;

aio_read(&cb);  // Asynchronous read call

// You can perform other tasks here while the read operation is ongoing
```

Blocking open

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

Asynchronous I/O (POSIX AIO)

Example:

Non-blocking open

```
int fd = open("example.txt", O_RDONLY | O_NONBLOCK);
```

```
Note that this flag has no effect for regular files and block devices;
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Blocking and Non-blocking System Calls

Comparison of Blocking and Non-Blocking Approaches

Blocking I/O

- Simpler code flow.
- Easier to implement in cases where performance or responsiveness isn't critical.
- Program execution pauses until the I/O operation is complete.

Non-blocking I/O:

- Prevents program from being stalled while waiting for I/O.
- Requires more complex logic (e.g., checking if the operation is complete or using event loops).
- Useful for applications that need to stay responsive (like GUIs or network servers).

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.